

GitGuardian Client Credentials Exposure Remediation Guide

1. What happened?

If GitGuardian detected a Client ID/Client Secret in your automation repository, treat the Client Secret as compromised. Removing it only from the latest source file does not remove it from Git history.

2. Do you need a new Client ID?

Usually, no. If only the Client Secret was exposed, keep the existing Client ID and regenerate/rotate the Client Secret. If your security policy requires a completely new application identity, create a new OAuth client, which gives you a new Client ID and a new Client Secret.

3. Keycloak — rotate the Client Secret

1. Open the Keycloak Admin Console.
2. Select the affected Realm.
3. Go to **Clients** and open the affected client.
4. Open **Credentials**.
5. Use **Regenerate** for the Client Secret.
6. Copy the new secret and store it securely.
7. Update the automation CI/CD secret store.
8. Run the API automation using the new credentials.
9. Verify that the old credential no longer works.

4. If a new Client ID is required

Create a new client in Keycloak (or your organization's Identity Provider), configure the required OAuth flow and permissions, and let the Identity Provider register the new Client ID and generate the new Client Secret.

New Client → New Client ID + Secret → Update CI/CD → Test → Disable old Client

5. Remove credentials from source code

Never hardcode the Client Secret in Java, Python, Postman collections, or configuration files.

Bad:

```
String clientSecret = "OLD_EXPOSED_SECRET";
```

Good:

```
String clientId = System.getenv("CLIENT_ID");  
String clientSecret = System.getenv("CLIENT_SECRET");
```

6. Store the new secret securely

Platform	Recommended storage
----------	---------------------

Jenkins	Jenkins Credentials
GitHub Actions	GitHub Actions Secrets
GitLab	CI/CD Variables
Azure DevOps	Secret Variables / Key Vault
AWS	AWS Secrets Manager

7. Clean the Git history

Deleting the secret from the current file is not enough if it exists in previous commits. Coordinate with your repository/security owners before rewriting shared history. A common modern approach is **git-filter-repo**.

Example pattern (do not put a real secret in your shell history):

```
git clone --mirror <repository>
cd <repository>.git
git filter-repo --path <affected-file> --invert-paths
git push --force --mirror
```

The exact cleanup command depends on where the credential was committed. Follow your organization's approved GitGuardian/security remediation process.

8. Recommended order of remediation

1. Revoke/rotate the exposed Client Secret
↓
2. Generate the replacement credential
↓
3. Update CI/CD secret storage
↓
4. Remove hardcoded credentials from source
↓
5. Remove the secret from Git history where required
↓
6. Push the cleaned repository using your approved process
↓
7. Run GitGuardian/security scan again
↓
8. Verify automation with the new credential

9. Final secure architecture

```
Git Repository
↓
Automation Framework
↓
CI/CD Secret Store
↓
CLIENT_ID + CLIENT_SECRET
↓
Authorization Server / Keycloak
```

↓
Access Token
↓
Protected API

10. Security checklist

- Assume an exposed Client Secret is compromised.
- Rotate/revoke the old secret first.
- Never reuse the exposed secret.
- Never commit the replacement secret.
- Do not print secrets or access tokens in CI logs.
- Use separate credentials for QA, UAT, and Production.
- Use least-privilege scopes/roles.
- Coordinate history rewriting with repository owners.
- Confirm the old credential no longer works.
- Rescan with GitGuardian after remediation.

Short answer

GitGuardian finds your secret → immediately rotate/revoke it → generate a new secret (or a new Client ID + Secret if required) → update CI/CD secrets → remove the old secret from Git history → rescan → verify automation.